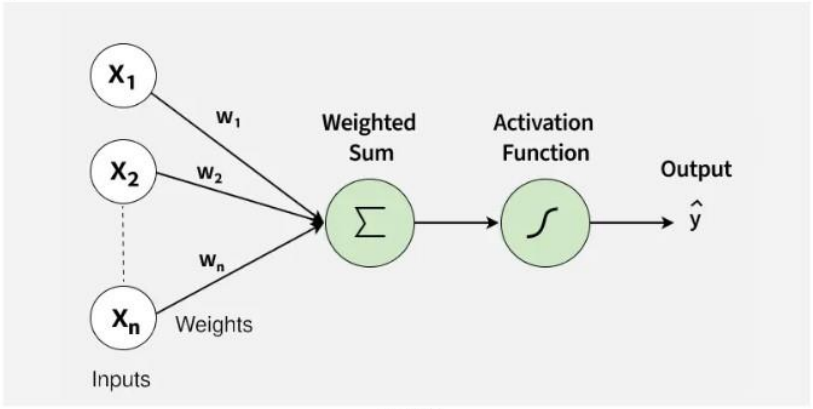
**What is Perceptron**

A Perceptron is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It is mainly used for binary classification problems. It forms the basic building block of many deep learning models.

- Takes multiple inputs and assigns weights
- Computes a weighted sum and applies a threshold
- Outputs either 0 or 1 (binary outcome)
- Forms the foundation of larger neural networks

Core Components




Perceptron

**1. Inputs (x1,x2,...,xn)**

These are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.

- **Example**: For an OR gate, the inputs are binary: $(x1,x2) \in \{0,1\}2$
- Inputs themselves have no inherent influence unless multiplied by weights.

**2. Weights (w1,w2,...,wn)**

Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.

- Weights are learned during training, adjusting based on errors.
- They act like importance scores for each feature.

**3. Bias (b)**

The bias is a constant value added to the weighted sum to shift the decision boundary.

- It allows the perceptron to classify correctly even when all input features are zero.
- Bias ensures the model is not forced to pass the decision boundary through the origin.

**Difference Between Weights and Bias**

- Weights control how much each input influences the output.
- Bias controls when the perceptron activates, independent of any input.

- Mathematically, Weights tilt the line and Bias shifts the line up/down or left/right.

**4. Net Input (Weighted Sum)**

This is the combined effect of all inputs and their weights:

$$z = \sum_{i-1}^{n} w_i x_i + b$$

- Represents the activation strength before passing through the activation function.
- If *zz* is high or low enough, it determines the final class.

**5. Activation Function (Step Function)**

The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- Output is always 0 or 1 making perceptrons suitable for binary classification.

Multi-layer Perceptron

An artificial neural network (ANN), often known as a neural network or simply a neural net, is a machine learning model that takes its cues from the structure and operation of the human brain. It is a key element in machine learning's branch known as deep learning. Interconnected nodes, also referred to as artificial neurons or perceptrons, are arranged in layers to form neural networks. An input layer, one or more hidden layers, and an output layer are examples of these layers. A neural network's individual neurons each execute a weighted sum of their inputs, apply an activation function to the sum, and then generate an output. The architecture of the network, including the number of layers and neurons in each layer, might vary significantly depending on the particular task at hand. Several machine learning tasks, such as classification, regression, image recognition, natural language processing, and others, can be performed using neural networks because of their great degree of versatility.

In order to reduce the discrepancy between expected and actual outputs, a neural network must be trained by changing the weights of its connections. Optimization techniques like gradient descent are used to do this. In particular, deep neural networks have made significant advances in fields like computer vision, speech recognition, and autonomous driving. Neural networks have demonstrated an exceptional ability to resolve complicated issues. They play a key role in modern AI and machine learning due to their capacity to automatically learn and extract features from data.

Supervised Neural Network models

A supervised neural network model is a type of machine learning model used for tasks where you have labelled data, meaning you know both the input and the corresponding correct output. In this model, you feed input data into layers of interconnected artificial neurons, which process the information and produce an output. During training, the model learns to adjust its internal parameters (weights and biases) to minimize the difference between its predictions and the actual labels in the training data. This process continues until the model can make accurate predictions on new, unseen data. Supervised neural

networks are commonly used for tasks like image classification, speech recognition, and natural language processing, where the goal is to map inputs to specific categories or values.

Multi-Layer Perceptron Architecture

MLP (Multi-Layer Perceptron) is a type of neural network with an architecture consisting of input, hidden, and output layers of interconnected neurons. It is capable of learning complex patterns and performing tasks such as classification and regression by adjusting its parameters through training. Let's explore the architecture of an MLP in detail:

- Input Layer: The input layer is where the MLP and dataset first engage with one another. A feature in the incoming data is matched to each neuron in this layer. For instance, each neuron might represent the intensity value of a pixel in picture categorization. These unprocessed input values are to be distributed to the neurons in the next hidden layers by the input layer.
- Hidden Layers: MLPs have a hidden layer or layers that are present between the input and output layers. The main computations happen at these layers. Every neuron in a hidden layer analyzes the data that comes from the neurons in the layer above it. In the same buried layer, neurons do not interact directly with one another but rather indirectly via weighted connections. The hidden layer transformation allows the network to learn intricate links and representations in the data. The intricacy of the task might affect the depth (number of hidden layers) and width (number of neurons in each layer).
- Output Layer: The MLP's neurons in the output layer, the last layer, generate the model's predictions. The structure of this layer is determined by the particular task at hand. The probability score for binary classification may be generated by a single neuron with a sigmoid activation function. Multiple neurons, often with softmax activation, can give probabilities to each class in a multi-class classification system. When doing regression tasks, the output layer frequently just has a single neuron that can forecast a continuous value.

Each neuron applies an activation function to the weighted total of its inputs, whether it is in the input, hidden, or output layer. The sigmoid, hyperbolic tangent (tanh), and rectified linear unit (ReLU) are often used activation functions. The MLP modifies connection (synapse) weights during training using backpropagation and optimization methods like gradient descent. In order to reduce the discrepancy between projected and actual outputs, this method aids the network in learning and fine-tuning its parameters. MLPs are appropriate for a variety of machine learning and deep learning problems, from straightforward to extremely complicated, due to their flexibility in terms of the number of hidden layers, neurons per layer, and choice of activation functions.

MLP Classifier with its Parameters

The MLP Classifier, short for Multi-Layer Perceptron Classifier, is a neural network-based classification algorithm provided by the Scikit-Learn library. It's a type of feedforward neural network, where information moves in only one direction: forward through the layers. Here's a detailed explanation of the MLP Classifier and its parameters, which in return collectively define the architecture and behavior of the MLP Classifier :

- Hidden Layer Sizes [Parameter: hidden_layer_sizes]: An MLP neural network's hidden_layer_sizes parameter is a crucial structural element. It describes how the network's hidden layers are structured. Each element of the tuple that this parameter accepts represents the number of neurons in a certain hidden layer. The network contains two hidden layers, with the first having 64 neurons and the second having 32 neurons, for instance, if hidden_layer_sizes is set to (64, 32). The network's ability to recognize complex patterns and correlations in the data is greatly influenced by the choice of the number of neurons and hidden

layers. In order to model complex data, deeper networks with more neurons may be more susceptible to overfitting.

- Activation Function [Parameter: activation]: Each neuron in the MLP's hidden layers is activated using a different activation function, which is determined by the activation parameter. The network can model intricate input-to-output mappings thanks to the non-linearity introduced by activation functions. There are numerous activation functions, each with their own special qualities. One well-liked option is "relu" (Rectified Linear Unit), which is both computationally effective and successful in reducing the vanishing gradient problem. Both "tanh" (Hyperbolic Tangent) and "logistic" (Logistic Sigmoid) are frequently used and have various uses.

- Solver for Weight Optimization [Parameter: solver]: The neural network's weights are updated during training using an optimization technique, which is determined by the solver parameter. To reduce the loss function of the network, several solvers use various methods. The "adam" algorithm, which combines ideas from RMSprop and Momentum, works well with large datasets and intricate models. The limited-memory Broyden-Fletcher-Goldfarb-Shanno optimization algorithm is used by "lbfgs," which is best for smaller datasets. The stochastic gradient descent algorithm, known as "sgd," adjusts weights based on random selections (mini-batches) of the training data at each iteration.

- Learning Rate [Parameter: learning_rate]: Each training iteration's weight updates are controlled by the learning_rate parameter. It is essential in establishing the training process's stability and rate of convergence. The learning rate might be "constant," "invscaling," or "adaptive," which can all have an impact on how it changes over time. It is crucial to choose the right learning rate since an extremely high rate might cause divergence or slow convergence, while a rate that is too low can cause very slow convergence.

- Maximum Iterations [Parameter: max_iter]: The max_iter parameter restricts how many iterations can be made before the solver converges during training. Convergence is the condition at which further iterations of the network's weights do not appreciably lower the loss. The solution quits and returns the current findings, which might not be ideal, if it cannot converge within the predetermined limit. When max_iter is chosen appropriately, the training process is neither abruptly stopped nor overly prolonged, allowing the model to converge to the desired level.

Boolean Function Implementation Using Perceptron

**What is a Boolean Function?**

A **Boolean function** takes binary inputs and gives a binary output.

The values are usually:

- **0 = False**
- **1 = True**

Common Boolean functions are:

- AND
- OR
- NOT
- XOR

## OR Function

- The textbook uses the **OR function** as a Perceptron example.

## OR Truth Table

| $X_1$ | $X_2$ | Target |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Rule of OR**

If **at least one input is 1**, the output is 1.

Only:

**0 OR 0 = 0**

All other combinations give **1**.

**How does a Perceptron learn OR?**

The Perceptron receives:

**Input + Target**

For example:

Input: 0 1

Target: 1

The Perceptron calculates its output using:

$Yin=b+X1W1+X2W2$

where:

- $\mathbf{X_1, X_2}$ → input values
- $\mathbf{W_1, W_2}$ → weights
- **b** → bias

Then an activation function gives the final output.

**What happens during training?**

The Perceptron follows these steps:

Input

↓

Calculate output

↓

Compare with target

↓

Is output correct?

↓

Yes → No weight change

↓

No → Update weights

↓

Repeat

The Perceptron continues this process until it learns the correct outputs.

**Training Examples**

The four training examples for OR are:

| Training Example | Input | Target |
|---|---|---|
| 1 | 0, 0 | 0 |
| 2 | 0, 1 | 1 |
| 3 | 1, 0 | 1 |
| 4 | 1, 1 | 1 |

These examples are given to the Perceptron during training.

**Weight Update**

If the Perceptron produces the wrong output, its weights are changed.

The textbook Perceptron learning rule is:

$$W_i(new) = W_i(old) + \alpha t X_i$$

and

$$b(new) = b(old) + \alpha t$$

where:

- $\boldsymbol{\alpha}$ = learning rate
- $\mathbf{t}$ = target
- $\mathbf{X_i}$ = input
- $\mathbf{W_i}$ = weight

The purpose of changing the weights is to make the Perceptron give a better output next time.

**Why can Perceptron learn OR?**

OR is a **linearly separable Boolean function**.

That means we can separate the output classes using a single decision boundary.

Therefore:

**A single-layer Perceptron can learn the OR function.**

**MATLAB OR Implementation**

The textbook also gives an example of implementing the OR function using the **MATLAB Neural Network Toolbox**.

The input data are:

data1 = [0 0 1 1

0 1 0 1]

These represent:

0 0

0 1

1 0

1 1

The target data are:

data2 = [0 1 1 1]

So the network has to learn:

0 0 → 0

0 1 → 1

1 0 → 1

1 1 → 1

**Perceptron Network in MATLAB**

In the textbook example, a **Perceptron network** is created.

Important settings include:

- **Network type:** Perceptron
- **Transfer function:** Hardlim
- **Learning function:** Learnp
- **Input:** OR input data
- **Target:** OR output data

The network is then trained using the training data.

**Result**

After training, the network is simulated using the same input data.

The expected output is:

[0 1 1 1]

This means:

0 OR 0 = 0

0 OR 1 = 1

1 OR 0 = 1

1 OR 1 = 1

So the Perceptron has successfully **learned the OR Boolean function**.

Training Algorithm

Adaline is an Adaptive Linear Neuron that learns by adjusting its weights based on the error between the target output and the calculated output.

The Adaline network training algorithm is as follows:

Step 0: Weights and bias are set to some random values but not zero. Set the learning rate parameter $\alpha$.

Step 1: Perform Steps 2–6 when stopping condition is false.

Step 2: Perform Steps 3–5 for each bipolar training pair $s:t$.

Step 3: Set activations for input units $i = 1$ to $n$.

$$x_i = s_i$$

Step 4: Calculate the net input to the output unit.

$$y_{in} = b + \sum_{i=1}^{n} x_i w_i$$

Step 5: Update the weights and bias for $i = 1$ to $n$:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha (t - y_{in}) x_i$$
$$b(\text{new}) = b(\text{old}) + \alpha (t - y_{in})$$

Step 6: If the highest weight change that occurred during training is smaller than a specified tolerance then stop the training process, else continue. This is the test for stopping condition of a network.

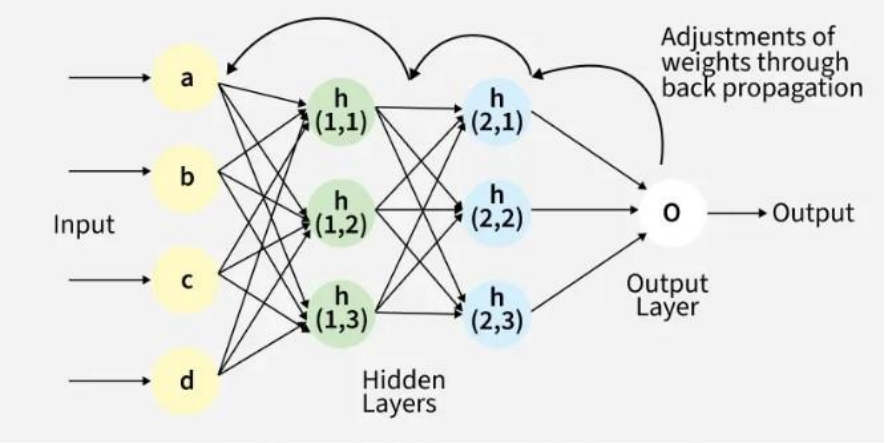
The range of learning rate can be between 0.1 and 1.0.

**Backpropagation in Neural Network**

Backpropagation is an algorithm that trains neural networks by reducing prediction error. It works by propagating errors backward, computing gradients using the chain rule, and updating weights and biases to improve performance.

- Computes gradients of the loss function with respect to each weight using the chain rule
- Updates weights and biases efficiently to reduce error
- Scales well to deep and complex neural networks

- Enables automatic learning and continuous improvement across training iterations




Fig(a) A simple illustration of how the backpropagation works by adjustments of weights

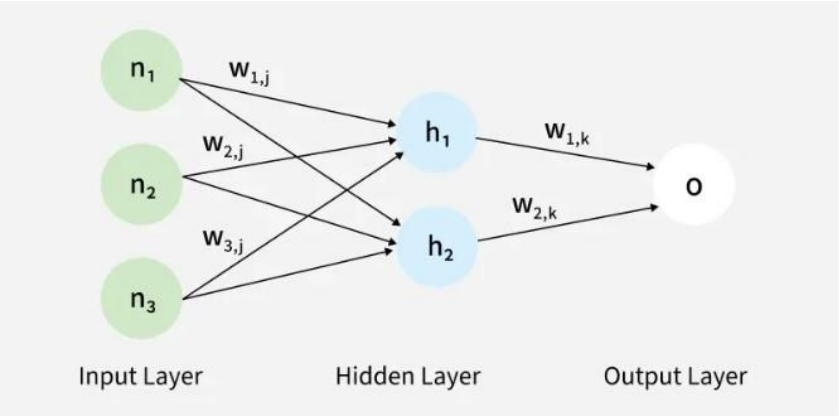
**Working of Back Propagation Algorithm**

The Back Propagation algorithm involves two main steps:

**1. Forward Pass Work**

In forward pass, input data moves through the network to generate an output.

- Input data is passed to the input layer and forwarded to hidden layers
- Each neuron computes a weighted sum and adds a bias
- Activation functions (like ReLU) are applied to produce outputs
- Outputs from one layer become inputs to the next layer
- Final layer uses functions like softmax to produce predictions (e.g., probabilities for classification)




The forward pass using weights and biases

**2. Backward Pass**

In this step, the error between predicted and actual output is propagated backward to update weights and biases. Error is calculated (e.g., using Mean Squared Error)

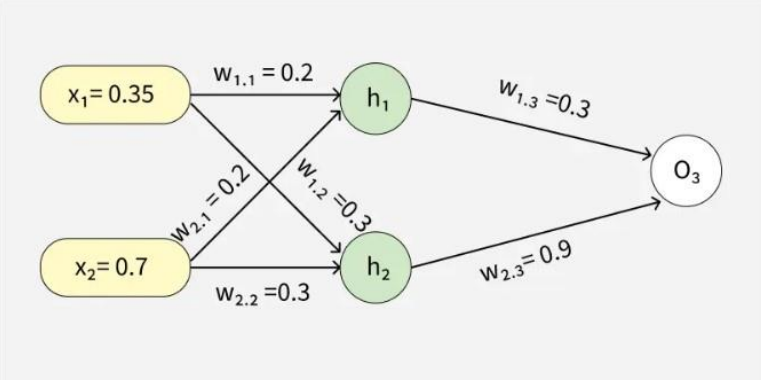
MSE=(Predicted Output−Actual Output)2MSE=(Predicted Output−Actual Output)2

- Gradients are computed using the chain rule

- Weights and biases are updated to reduce the error
- Process continues layer by layer from output to input
- Derivatives of activation functions are used to compute gradients effectively

## Example

- Let’s walk through an example of Back Propagation in machine learning. Assume the neurons use the sigmoid activation function for the forward and backward pass. The target output is 0.5 and the learning rate is 1.




Example (1) of backpropagation sum